Lab1 report

Name: 박수현

Student ID: 2022063672

Class Code: 12916

1. Implementation

The ALU (Arithmetic Logic Unit) module consists of four input buses and one output bus.

- Operand1 & Operand2: 32-bit buses for data operands.

- Shamt: 5-bit bus for specifying the shift amount.

- Funct: 4-bit bus to determine the specific ALU operation.

- alu_result: 32-bit ALU operation result

The specific code for calculation is almost the same as C, so, I will not explain it in this report. However, handling signed value and arithmetic right shift operation is slightly different in Verilog. In Verilog, If the variable is not defined as signed explicitly, the value would be treated as unsigned value. Furthermore, if the '>>>' operator is applied to an unsigned value, Verilog treats it as a logical right shift. Similarly, the '<' operator behaves as an unsigned comparison unless both operands are explicitly defined as signed types.

HW01 Skeleton Code > V ALU.v

```
1  `timescale 1ns / 1ps
2  `include "GLOBAL.v"
3
4  // this is a combinational logic
5  module ALU(
6          input [31:0]        operand1,
7          input [31:0]        operand2,
8          input [4:0]         shamt,
9          input [3:0]         funct,
10         output reg [31:0]   alu_result
11     );
12
13     // FIXME
14     always @(*) begin
15         alu_result = 0;
16         case (funct)
17             `ALU_ADDU: alu_result = operand1 + operand2;
18             `ALU_AND : alu_result = operand1 & operand2;
19             `ALU_NOR : alu_result = ~(operand1 | operand2);
20             `ALU_OR  : alu_result = operand1 | operand2;
21             `ALU_SLL : alu_result = operand2 << shamt;
22             `ALU_SRA : alu_result = $signed(operand2) >>> shamt;
23             `ALU_SRL : alu_result = operand2 >> shamt;
24             `ALU_SUBU: alu_result = operand1 - operand2;
25             `ALU_XOR : alu_result = operand1 ^ operand2;
26             `ALU_SLT : alu_result = $signed(operand1) < $signed(operand2);
27             `ALU_SLTU: alu_result = operand1 < operand2;
28             `ALU_EQ  : alu_result = operand1 == operand2;
29             `ALU_NEQ : alu_result = operand1 != operand2;
30             `ALU_LUI : alu_result = operand2 << 16;
31         endcase
32     end
33 endmodule
```

2. Troubleshooting

In my first attempt, I implemented the arithmetic right shift operation as alu_result = operand2 >>> shamt. However, it functioned as a logical right shift instead of arithmetic right shift. I later found that since the default data type in Verilog is unsigned, I needed to explicit type cast the operand to a signed type to perform arithmetic right shift.